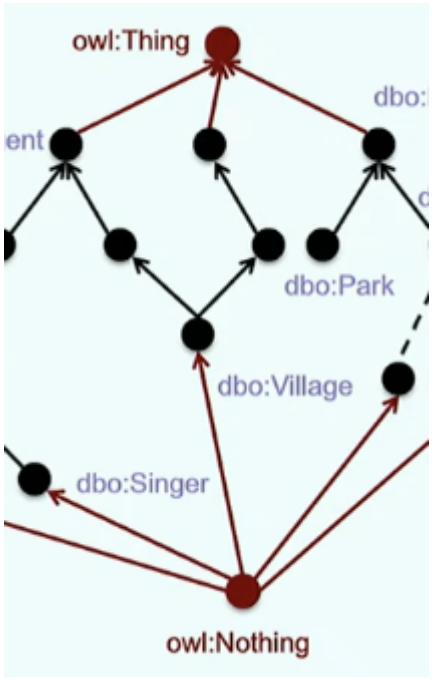
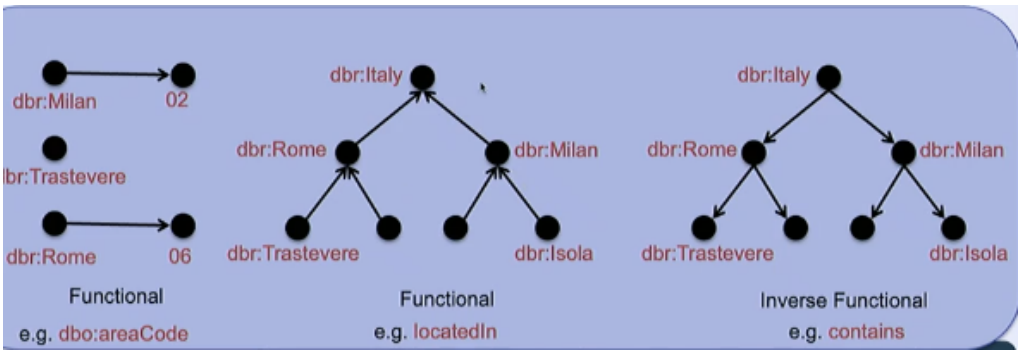
- E' diviso in 3 linguaggi
  - OWL 2 EL: supporta ragionamenti molto veloci
  - OWL 2 QL: OWL 2 però possibile la traduzione per database relazionali
  - OWL 2 RL: usato per essere implementato in sistemi a regole
- Assunzioni:
  - Open world assumption: ciò che non è dimostrabile dalla knowledge base non è assunta falsa
    - (la closed invece assume falsa)
    - OWL2 usa la open, per closed dobbiamo specificare
  - No unique names assumption: nomi diversi non assumono entità diverse ameno che esplicitato
    - (la unique invece nomi invece nomi diversi sono entità diverse)
      - Firenze != Pisa
      - In owl lo dobbiamo dire esplicitamente
    - OWL usa la no unique names
- OWL c'è una distizione
  - ObjectProperties: i valori devono essere delle risorse (iri o blank nodes)
    - Rappresentazione relazioni
    - Ci sono degli assiomi su di esse:
      - Simmetria:
        - Se io sono fratello di marco, allora marco è fratello mio
      - Inversa:
        - Qui è simmetria però con 2 proprietà inverse:
        - Se io sono figlio di Antonella, allora Antonella è madre mia
      - Transitività
        - Se io sono fratello di luca, e luca è fratello di marco, allora io sono fratello di marco
      - Functional
        - Io non posso avere 2 madri, e se dico che mia madre è sia maria che anna, allora lui penserà che maria = anna
        - Però maria può essere madri di più persone
        - Oppure persino anche un codice fiscale (non tutte le persone hanno un codice fiscale)
        - Non può essere funzionale a transitivo
        - Qui posso avere al massimo 1 arco uscente
        - Il nodo sopra, dbr:Italy viene chiamato owl:Thing, la classe che ha tutto

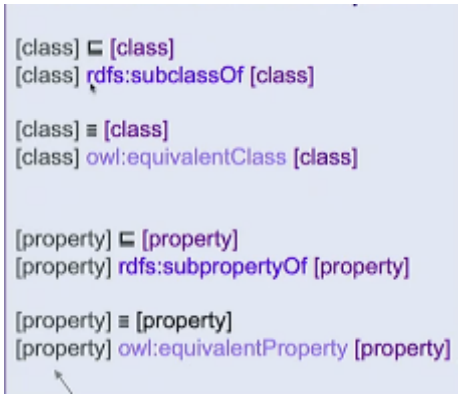
        - Invece le foglie sono collegate a un owl:nothing
            - Non potrà mai avere un istanza
        - Così facendo abbiamo un reticolo
            - Che è importante siccome un reticolo ci permette di usare and, or e not
        - 

        - L'autologia **non** è un albero è un reticolo
    - Inverse functional
        - L'opposto del functional, mamma può essere madre di più persone
        - 

        - Aka li posso usare per fare alberi, o albero famigliare
        - Qui posso avere al massimo 1 arco entrante
- DatatypeProperties: i valori devono essere literali
    - Rappresentano attributi
- Cos'è una base di conoscenza in conoscenze descrittive
    - Fatta in 2 blocchi
        - Terminologico riguardante le relazioni
            - Parliamo delle classi e delle proprietà
            - Tbox = Terminology box

-
    - Notiamo: relazioni
- Rbox = relational box
- In esame però di solo tbox, siccome così devi imparare di meno

Struttura del Tbox rispetto all'rdfs:



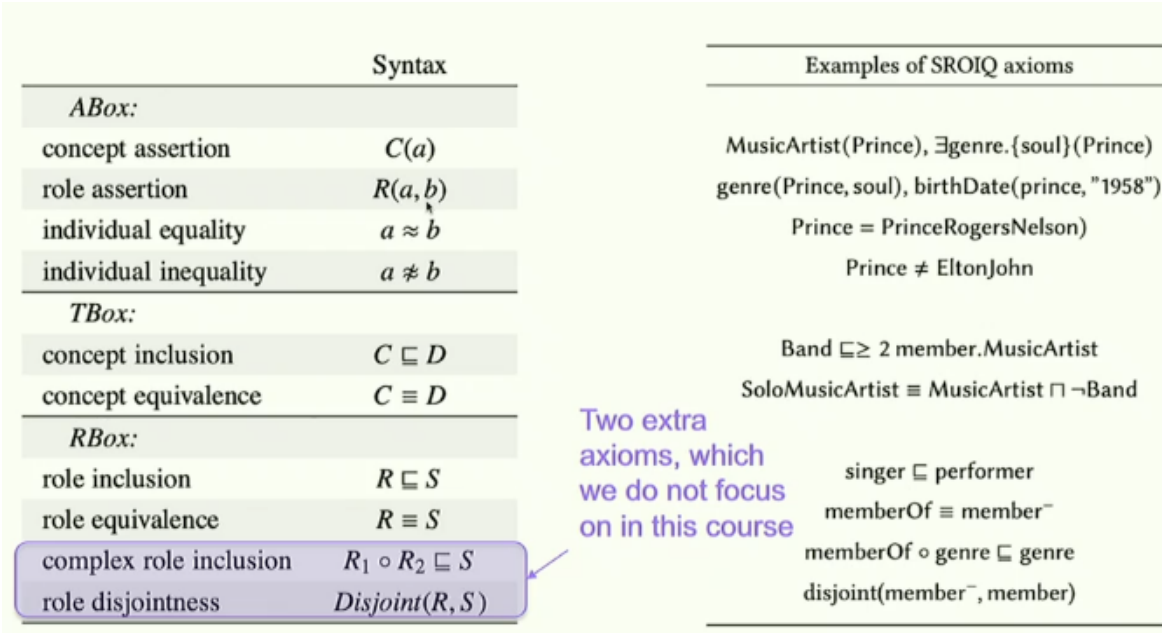
-
    - Rispetto all'rdfs, possiamo anche dire che una classi è equivalente
    - Questi sono tutti gli assiomi di sottoclassi o equivalenza
    - Es:
        -
            - equivalentclass
            - Se qualcuno è una persona e una femmina, allora è una donna, e viceversa
            - Specifica condizioni necessarie
        -
            - subclassof
            - Se x è una donna, allora è una persona e una femmina
            - Se qualcuno è persona e femmina, non possiamo dedurre subito che sia una donna
            - Specifica condizioni necessarie e sufficenti
        -
        -
            - Questa è la disgiunzione
        - =
            - owl:disjointWith
            - Nessuno può appartenere ad entrambe le classi
        -
        -
            - I film fanno parte, tutti i flm, di qualcosa che ha come produttore movie produttore
        -
            - Ogni album ha almeno qualche artists che è un music artists
        -

-
    - Vincoli di cardinalità
    - Un gruppo è composto da almeno 2 music artists
    - Si potrebbe togliere MusicArtists, così facendo deve essere composto da "2 persone generiche"
- :equivalentProperty
    - E' come se facessimo degli alias, se diciamo che i beatles member sono member, allora li possiamo trovare come member

- Dati
    - Parliamo degli individui
    - Abox = Assertion box
    - Abox fatta con assiomi con questa struttura::
        - Predico un istanza fa parte di una classe
          $n\ \ a\ \ ate$
          1
        - Predico che 2 istanze sono in relazione tra loro
          ,
- Una interpretazione soddisfa:
    - T/Abox sse soddisfa ogni assioma nella T/Abox
      Affinchè questo sia vero, devono essere corretti:
        - Soddisfabilità, un concetto C soddisfa una Tbox se esiste almeno 1 istanza tale che C non è vuoto
          Parent intersecato a non person non è soddisfacibile siccome è insieme vuoto
        - Sussunzione, un concetto C è sussunto da un concetto D se in tutte le soddisfazione di C sono sottoinsieme di D
          C deve essere sottoclasse anche di D
    - Se una interpretazione soddisfa sia T che A box, allora soddisfa anche la knowledge base
        - La conoscenza è corretto
            - Se D sussume C
        - La conoscenza è minimamente redundante
            - Se C è equivalente di D e non cattura riduzioni
        - La conoscenza è significativa
            - Se C è insoddisfacibile allora non ha alcun senso

- OWL può usare la description logic
    - Pro:
        - Semantica ben definita

    - Proprietà formali formalizzate
    - Algoritmi di reasoning conosciuti
    - Sistema ottimizzato
- Differenze di nomi:
    - Classe => Concetto
    - Proprietà / Relazione => Ruolo
    - Istanza / Entità => Individual
        - Fanno parte della sintassi
- Assiomi della A-T-R box



	Syntax
ABox:	
concept assertion	$C(a)$
role assertion	$R(a,b)$
individual equality	$a \approx b$
individual inequality	$a \not\approx b$
TBox:	
concept inclusion	$C \sqsubseteq D$
concept equivalence	$C \equiv D$
RBox:	
role inclusion	$R \sqsubseteq S$
role equivalence	$R \equiv S$
complex role inclusion	$R_1 \circ R_2 \sqsubseteq S$
role disjointness	$Disjoint(R,S)$

Examples of SROIQ axioms
MusicArtist(Prince), ∃genre.{soul}(Prince)
genre(Prince, soul), birthDate(prince, "1958")
Prince = PrinceRogersNelson)
Prince ≠ EltonJohn
Band ⊑≥ 2 member.MusicArtist
SoloMusicArtist ≡ MusicArtist ⊓ ¬Band
singer ⊑ performer
memberOf ≡ member⁻
memberOf ∘ genre ⊑ genre
disjoint(member⁻, member)

Two extra axioms, which we do not focus on in this course

    - Gli assiomi sono interpretati come modifiche in funzioni booleane (vero/falso)
        - Gli assiomi possono essere anche inferiti di altri assiomi (premesse)
- Semantica
    - E' data da 1 funzione di interpretazione del linguaggio
        - $I = {}^{I}, {}^{I}$
            - ^{I} E' il dominio
            - ^{I} è una funzione di interpretazione che mappa:
                - Ogni concetto atomico C che è un insieme del dominio
                - Ciascun ruolo atomico è mappato tra tutte le possibile coppie che possiamo costruire nel dominio (Aka tutte combinazioni di ^{I})
                - Ciascuna costante a è mappato nel dominio
                - La funzione di interpretazione è estesa ricorsivamente per ogni concetto
- OWL e le DL assumono una semantica del mondo aperto